GIÁO TRÌNH THỰC HÀNH · LẬP TRÌNH BACKEND

# CHƯƠNG 05

# MIDDLEWARES & VALIDATION

Bài thực hành Buổi 06 — Hệ Liên thông

**NÂNG CẤP API QUẢN LÝ SINH VIÊN TỪ CHƯƠNG 04**

NODE.JS + EXPRESS + TYPESCRIPT + PNPM

ZOD · MORGAN · OPENAPI 3.0 YAML · POSTMAN · CTCO



TRANG ĐỊNH HƯỚNG

# 01 · Mục tiêu và điều kiện chuẩn bị

Bài này nối trực tiếp Chương 04: RESTful CRUD và Swagger. Phần trọng tâm là kiểm tra dữ liệu đầu vào, hiểu luồng Middleware, ghi log và review code AI.

## Hoàn thành bài thực hành, sinh viên có thể

- **Giải thích** req, res, next() và thứ tự Route  Middleware  Controller  Service.
- **Viết Zod Schema** kiểm tra POST/PUT của API SinhVien đã có.
- **Viết Middleware validate(schema)** dùng chung, từ chối dữ liệu sai trước Controller.
- **Ghi access.log** bằng Morgan, hiểu cơ chế ghi nối (append).
- **Kiểm thử** trên Postman/Swagger; cập nhật YAML và giữ nguyên chức năng Chương 04.
- **Viết và phản biện prompt CTCO**, ghi lại thay đổi, kết quả kiểm thử và giải thích code.

## Sinh viên phải có sẵn

| Bắt buộc | Kiểm tra nhanh |
|---|---|
| Project **backend-quan-ly-sinh-vien** đã hoàn thành Chương 04 | Có package.json, các API CRUD và docs/openapi.yaml |
| Node.js 20+, pnpm, VS Code, Postman | node -v / pnpm -v |
| Nguồn dữ liệu hiện tại | Giữ SQL Server + Sequelize nếu đã dùng; giữ RAM nếu bài cũ dùng RAM |
| Swagger UI đã chạy | Mở /api-docs trên đúng PORT hiện tại |

> **Phạm vi của Buổi 06**
>
> Middleware + Validation bằng **Zod**; Logging bằng **Morgan**; chỉ phân tích logic kiểm tra Role. **Không làm đăng nhập, JWT, bcrypt hoặc hệ thống phân quyền thật**; các nội dung đó sẽ học ở Buổi 07.

KIẾN THỨC NỀN

# 02 · Middleware nằm ở đâu?

Middleware là hàm trung gian trong Express, nhận **req**, **res** và **next()**. Nó có thể kiểm tra, ghi nhận hoặc kết thúc Request trước khi tới Controller.

## Luồng xử lý mới của API SinhVien




## Hai kết quả của Validation Middleware

| Khi hợp lệ | Khi không hợp lệ |
|---|---|
| Zod **safeParse()** trả success=true; Middleware gán dữ liệu đã chuẩn hóa vào req.body rồi gọi **next()**. | Middleware trả **HTTP 400** với JSON báo lỗi; **không gọi next()**; Controller không chạy. |
| Ví dụ: POST đủ maSV, hoTen, email, maLop. | Ví dụ: POST email sai, thiếu hoTen hoặc có trường ngoài hợp đồng. |

## Phân biệt ba khái niệm

- **Validation**: xác nhận hình dạng/kiểu/giá trị cơ bản của Request (bài này).
- **Authentication**: xác minh người dùng là ai (JWT ở Buổi 07).
- **Authorization**: kiểm tra người dùng đã xác thực có quyền hay không (phát triển từ Buổi 07).

> **Điều cần nhớ**
>
> Swagger YAML **mô tả** dữ liệu hợp lệ nhưng **không tự ngăn** dữ liệu sai ở runtime. Chương 05 tạo Validation Middleware thực thi quy tắc đó trong Express.

ÔN TẬP CHƯƠNG 04

# 03 · Hợp đồng API phải giữ nguyên

Trước khi sửa, đọc lại quy ước trong bài RESTful CRUD & Swagger. Mọi bổ sung Validation phải tương thích với các Request/Response đã được kiểm thử.

| Phương thức | Endpoint | Mong đợi |
|---|---|---|
| GET | /api/v1/sinh-vien | 200 · danh sách (data là mảng) |
| GET | /api/v1/sinh-vien/:maSV | 200 hoặc 404 |
| POST | /api/v1/sinh-vien | 201 hoặc 400/409 |
| PUT | /api/v1/sinh-vien/:maSV | 200 hoặc 400/404 |
| DELETE | /api/v1/sinh-vien/:maSV | 200 hoặc 404 |

## Quy tắc dữ liệu đầu vào

- **POST** yêu cầu: maSV, hoTen, email, maLop. Mã sinh viên phải duy nhất.
- **PUT** lấy maSV từ URL; body chỉ gồm hoTen, email, maLop (đủ cả ba).
- Chuỗi phải không rỗng sau **trim()**; email phải có định dạng cơ bản hợp lệ.
- Không chấp nhận thuộc tính lạ trong body. Giữ giới hạn độ dài/khóa ngoại đang có trong SQL Server.

## Response lỗi tương thích Chương 04

```
{
  "success": false,
  "message": "Dữ liệu đầu vào không hợp lệ",
  "data": null,
  "errors": [{ "field": "email", "message": "Email không hợp lệ" }]
}
```

Trường **errors** là phần mở rộng **tùy chọn** trong bài này để sinh viên biết chính xác thuộc tính sai. Ba trường success/message/data vẫn được giữ. Cần cập nhật OpenAPI để phản ánh trường mới.

> **Giữ nguyên tầng nghiệp vụ**
>
> Zod kiểm tra cấu trúc Request, nhưng **Service/Database** vẫn phải kiểm tra mã trùng (409), bản ghi không tồn tại (404) và các ràng buộc nghiệp vụ/khóa ngoại. Không bỏ kiểm tra nghiệp vụ chỉ vì có Validation.

BẢN ĐỒ PROJECT

# 04 · Cây thư mục và chính xác file cần sửa

Không chạy npm init, không tạo backend-buoi06. Các đường dẫn có thể khác ở dự án thực tế; đối chiếu trước khi yêu cầu AI sửa.

```
backend-quan-ly-sinh-vien/
├── package.json                # GIỮ NGUYÊN
├── pnpm-lock.yaml              # CẬP NHẬT KHI CÀI THƯ VIỆN
├── .env                        # KHÔNG CHIA SẺ
├── .gitignore                  # BỔ SUNG logs/*.log nếu thiếu
├── src/
│   ├── app.ts / server.ts      # SỬA FILE EXPRESS ĐANG CÓ
│   ├── config/swagger.config.ts # GIỮ NGUYÊN
│   ├── routes/sinhVien.routes.ts # THÊM validate vào POST/PUT
│   ├── controllers/...         # GIỮ NGUYÊN
│   ├── services/...            # GIỮ NGUYÊN
│   ├── models/...              # GIỮ NGUYÊN
│   ├── middlewares/
│   │   ├── validate.middleware.ts # MỚI
│   │   └── access-log.middleware.ts # MỚI (Morgan)
│   └── validations/
│       └── sinhVien.schema.ts   # MỚI (Zod)
├── docs/
│   ├── openapi.yaml            # CẬP NHẬT MÔ TẢ VALIDATION
│   ├── prompt-log.md           # GHI NHẬT KÝ BUỔI 06
│   └── test-results.md         # GHI KẾT QUẢ TEST
├── postman/                    # GIỮ/EXPORT COLLECTION
└── logs/
    └── access.log              # MORGAN TỰ TẠO
```

**Lưu ý riêng với ESM/NodeNext**

Nếu Chương 04 đang import file nội bộ bằng đuôi **.js** trong TypeScript (ví dụ ../services/abc.js), hãy tiếp tục quy ước đó. **Không sửa tsconfig hoặc package.json module type một cách tùy tiện.**

BƯỚC 0 · TRƯỚC KHI THAY ĐỔI

# 05 · Kiểm tra project và lưu trạng thái ban đầu

Làm đúng bước này trước khi thêm thư viện. Nếu API gốc chưa chạy thì không thể biết lỗi phát sinh từ Chương 04 hay Chương 05.

## Thao tác Terminal tại thư mục có package.json

```
node --version
pnpm --version
pnpm install
pnpm build
pnpm dev
```

## Trong Postman hoặc Swagger, kiểm tra trước ba điểm

- GET /api/v1/sinh-vien  200, data là mảng.
- POST hợp lệ  201; PUT hợp lệ  200 (dùng sinh viên test, không sửa dữ liệu thật).
- Mở **/api-docs** trên PORT hiện tại và thấy danh sách endpoint.

### PROMPT 0 — KHẢO SÁT, CHƯA VIẾT CODE

**C — Context**: Tôi tiếp tục dự án quản lý sinh viên của Chương 04 bằng Express, TypeScript, pnpm; có CRUD và OpenAPI YAML.

**T — Task**: Đọc các file tôi cung cấp: package.json, tsconfig, app/server, Route, Controller, Service và OpenAPI. Xác định luồng POST.

**C — Constraint**: Không tạo project mới, không thay đổi file, không đọc/gửi bí mật từ .env; không phỏng đoán nội dung file chưa được cung cấp.

**O — Output**: Trả bảng “File — vai trò — giữ nguyên/sửa — lý do”, vị trí phù hợp để gắn Zod/Morgan và các rủi ro cần xác nhận.

### Điểm dừng bắt buộc

Nếu build hay một API nền bị lỗi, **chưa chuyển qua Bước 1**. Hãy sửa lỗi gốc và ghi lại kết quả trong docs/test-results.md.

BƯỚC 1 · QUAN SÁT REQUEST

# 06 · Hiểu req, res, next() bằng ví dụ ngắn

Bước này dùng Middleware đơn giản chỉ để minh họa. Sau khi thử xong có thể bỏ đoạn demo để tránh log lặp với Morgan.

## Ví dụ gắn tạm trước router trong app/server hiện có

```
// Thêm tạm trong file đã tạo Express app
app.use((req, _res, next) => {
  console.log(`[DEMO] ${req.method} ${req.originalUrl}`);
  next(); // Cho request tiếp tục đến router/controller
});
```

## Thực hiện thử nghiệm

- Mở Postman gọi GET /api/v1/sinh-vien. Quan sát Terminal có dòng [DEMO].
- Giải thích: req chứa thông tin Request, res dùng gửi Response, next() chuyển sang handler sau.
- Bỏ next() mà cũng không trả Response sẽ làm Request treo. Nếu đã trả lỗi bằng res.status(...).json(...), **không** gọi next() tiếp.

**PROMPT 1 — GIẢI THÍCH MIDDLEWARE**

**C — Context**: Tôi đã có API GET /api/v1/sinh-vien chạy được trong Express TypeScript.

**T — Task**: Giải thích bằng một Request thực tế vai trò req, res, next() và thứ tự app.use  router  controller.

**C — Constraint**: Chỉ minh họa, không thay đổi API hay dữ liệu; nêu hai tình huống khi gọi và không gọi next().

**O — Output**: Trình bày sơ đồ luồng ngắn, chú thích 5 dòng code minh họa và 3 câu hỏi để tự kiểm tra.

**Câu hỏi tự trả lời**

Nếu Validation phát hiện email sai, tại sao Middleware phải **return** sau khi gửi 400? Vì sao không nên để Controller xử lý dữ liệu sai rồi mới kiểm tra?

BƯỚC 2 · CÀI THƯ VIỆN

# 07 · Bổ sung Zod và Morgan vào dự án cũ

Zod giúp kiểm tra Request Body; Morgan ghi nhật ký HTTP Request. Không cài lại Express hoặc TypeScript nếu chúng đã có trong Chương 04.

## Cài package trong thư mục backend-quan-ly-sinh-vien

```
pnpm add zod morgan
pnpm add -D @types/morgan
```

Xem thư viện đã có trong package.json và chạy lại:

```
pnpm build
pnpm dev
```

## Hai thư viện làm gì?

| Thư viện | Chức năng trong bài thực hành |
|---|---|
| Zod | Định nghĩa **Schema**, thực thi **safeParse()**, trả các lỗi **issues**. |
| Morgan | Ghi log về method, URL, status; kết hợp stream để lưu vào **logs/access.log**. |
| @types/morgan | Cung cấp khai báo kiểu cho TypeScript nếu thư viện cần. |

Nếu thiếu pnpm: kiểm tra cách cài pnpm đã dùng ở Chương 04; không chuyển sang npm giữa chừng. Nếu bị lỗi quyền cài gói, kiểm tra Node và Terminal trước khi đổi cấu hình dự án.

**Điểm kiểm tra**

Sau bước này, **pnpm build** và **pnpm dev** phải chạy như trước; Swagger và 5 API cũ không thay đổi. Có thay đổi pnpm-lock.yaml là bình thường.

BƯỚC 3 · KIỂM TRA DỮ LIỆU

# 08 · Tạo Schema Zod cho POST và PUT

Tạo mới file **src/validations/sinhVien.schema.ts**. Body POST và PUT khác nhau nên không dùng cùng một Schema cho cả hai.

```
import { z } from 'zod';

const editableFields = {
  hoTen: z.string().trim().min(1, 'Họ tên không được rỗng'),
  email: z.string().trim().email('Email không hợp lệ'),
  maLop: z.string().trim().min(1, 'Mã lớp không được rỗng'),
};

export const createSinhVienSchema = z.object({
  maSV: z.string().trim().min(1, 'Mã SV không được rỗng'),
  ...editableFields,
}).strict();

export const updateSinhVienSchema = z.object({
  ...editableFields,
}).strict();
```

## Giải thích những lệnh quan trọng

- **z.string()**: bắt buộc đúng kiểu chuỗi; **trim()**: bỏ khoảng trắng hai đầu.
- **min(1)**: không cho chuỗi rỗng; **email()**: định dạng email cơ bản.
- **strict()**: từ chối body có thuộc tính ngoài hợp đồng.
- **...editableFields**: tái dùng các trường được phép sửa nhưng POST có thêm maSV.

> **Giới hạn kiểm tra**
>
> Zod không tự biết độ dài cột SQL Server hoặc khóa ngoại từ CSDL. Nếu dự án cũ có các giới hạn đó, cần giữ kiểm tra ở tầng dữ liệu/nghiệp vụ hoặc bổ sung Schema tương ứng sau khi đọc Model thực tế.

BƯỚC 4 · MIDDLEWARE TÁI SỬ DỤNG

# 09 · Viết validate(schema) và phản hồi lỗi 400

Tạo file **src/middlewares/validate.middleware.ts**. Schema được truyền vào Middleware nên cùng một hàm có thể dùng cho POST và PUT.

```
import type { RequestHandler } from 'express';
import type { z } from 'zod';

export const validate = (schema: z.ZodType): RequestHandler => {
  return (req, res, next) => {
    const parsed = schema.safeParse(req.body);

    if (!parsed.success) {
      res.status(400).json({
        success: false,
        message: 'Dữ liệu đầu vào không hợp lệ',
        data: null,
        errors: parsed.error.issues.map((issue) => ({
          field: issue.path.join('.') || 'body',
          message: issue.message,
        })),
      });
      return; // Dừng: không đi vào Controller
    }

    req.body = parsed.data; // Dùng dữ liệu đã trim
    next();                 // Hợp lệ: tiếp tục xử lý
  };
};
```

## Dòng code nào quyết định Request được đi tiếp?

- **safeParse** không tự gửi HTTP Response; Middleware phải kiểm tra success.
- **parsed.error.issues** tạo danh sách field/message giúp sửa lỗi đầu vào.
- **req.body = parsed.data** chuyển dữ liệu đã chuẩn hóa cho Controller.
- **return** sau khi gửi 400 ngăn gọi tiếp controller; **next()** chỉ chạy với dữ liệu đúng.

### Không thay đổi ngầm hợp đồng API

Giữ success/message/data từ Chương 04; chỉ thêm errors khi Validation thất bại. Nếu dự án đã có Middleware xử lý lỗi dùng chung, hãy đọc cấu trúc hiện tại và điều chỉnh để tránh hai định dạng Response khác nhau.

BƯỚC 5 · GẮN ĐÚNG VÀO ROUTER

# 10 · Tích hợp Middleware vào POST/PUT đã có

Mở **src/routes/sinhVien.routes.ts**. Không tạo router thứ hai. Giữ lại các handler và đường dẫn đang hoạt động trong Chương 04.

## Thêm import, theo đúng quy ước module của dự án

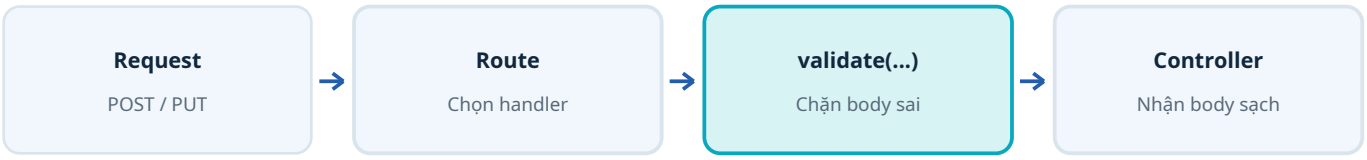
```
import { validate } from '../middlewares/validate.middleware.js';
import {
  createSinhVienSchema,
  updateSinhVienSchema,
} from '../validations/sinhVien.schema.js';
```

## Đổi đúng hai dòng khai báo Route (ví dụ tên handler Chương 04)

```
// TRƯỚC: router.post('/', taoSinhVien);
// SAU:
router.post('/', validate(createSinhVienSchema), taoSinhVien);

// TRƯỚC: router.put('/:maSV', capNhatSinhVien);
// SAU:
router.put('/:maSV', validate(updateSinhVienSchema), capNhatSinhVien);
```

## Vị trí của Middleware có ý nghĩa




- Các GET/DELETE giữ nguyên; Controller và Service không cần viết lại nếu không thay đổi nghiệp vụ.
- Router mount tại **/api/v1/sinh-vien** vẫn nằm trong Express app cũ.
- Nếu project dùng CommonJS hoặc tên handler khác, giữ import/handler phù hợp project; không copy nguyên khung rồi tạo route trùng.

**Kiểm tra ngay sau khi sửa**

Gửi POST thiếu hoTen  400; POST hợp lệ  201. Sau đó GET lại kiểm chứng. Nếu Swagger hoạt động nhưng API trả 404, kiểm tra mount prefix và tên Route, không sửa OpenAPI để che lỗi.

BƯỚC 6 · GHI NHẬT KÝ REQUEST

# 11 · Morgan ghi access.log với chế độ append

Tạo file **src/middlewares/access-log.middleware.ts**; ghi mỗi Request vào file để quan sát hoạt động hệ thống, không ghi password hoặc token.

```
import fs from 'node:fs';
import path from 'node:path';
import morgan from 'morgan';

const logsDir = path.resolve(process.cwd(), 'logs');
fs.mkdirSync(logsDir, { recursive: true });

const logFile = path.join(logsDir, 'access.log');
const logStream = fs.createWriteStream(logFile, { flags: 'a' });

export const accessLogger = morgan('combined', {
  stream: logStream,
});
```

### Trong file app.ts/server.ts đang tạo Express app, gắn một lần:

```
import { accessLogger } from './middlewares/access-log.middleware.js';

app.use(accessLogger); // Gắn trước các router để ghi Request
// app.use('/api/v1/sinh-vien', sinhVienRoutes); // GIỮ NGUYÊN
```

### Trong .gitignore (nếu chưa có):

```
logs/*.log
.env
node_modules/
dist/
```

- **flags: "a"** nghĩa là append: ghi nối vào cuối file khi server khởi động lại.
- Morgan “combined” ghi thông tin HTTP, có thể nhiều trường hơn phần minh họa; không nhất thiết lưu body.
- Tuyệt đối không đặt mật khẩu, JWT hoặc thông tin nhạy cảm trên URL/query vì URL có thể đi vào log.

BƯỚC 7 · SWAGGER YAML

# 12 · Cập nhật đặc tả OpenAPI sẵn có

Chỉ sửa **docs/openapi.yaml** hiện tại. Không tạo openapi.yaml mới ở thư mục gốc, không thêm JSDoc hoặc decorator @Swagger.

## Ý nghĩa cần cập nhật

- POST và PUT đã có response **400**: giữ nguyên tham chiếu BadRequest.
- Mô tả Response lỗi có thể kèm mảng errors để thể hiện lỗi từ Zod.
- Dữ liệu maSV/hoTen/email/maLop và trường required phải trùng với Schema của code.

## Trong components.schemas hiện có, thêm ValidationIssue ngang cấp SinhVien:

```
    ValidationIssue:
      type: object
      required: [field, message]
      properties:
        field: { type: string, example: email }
        message: { type: string, example: Email không hợp lệ }
```

## Trong ErrorResponse.properties đang có, thêm trường errors ngang cấp message/data:

```
        errors:
          type: array
          description: Chi tiết lỗi Validation (nếu có)
          items:
            $ref: '#/components/schemas/ValidationIssue'
```

### Kiểm chứng YAML

Đảm bảo chỉ có một **components**, một **schemas** và một **ErrorResponse**. Mở /api-docs; POST/PUT phải hiển thị đúng body, response 400 và vẫn “Try it out” được. YAML chỉ mô tả; Zod trong Route mới là nơi chặn Request thật.

BƯỚC 8 · KIỂM THỬ BẰNG POSTMAN

# 13 · Chuẩn bị Collection và dữ liệu test

Tiếp tục sử dụng Postman Collection của Chương 04. Không tạo API khác để kiểm thử. Đặt giá trị PORT đúng theo ứng dụng đang chạy (ví dụ 5000).

## Tạo hoặc giữ Environment có biến

| Biến | Giá trị minh họa | Mục đích |
|---|---|---|
| baseUrl | http://localhost:5000 | Không gõ lặp host/port trong mọi Request |
| maSV | SV006 | Mã dùng để thêm/sửa/xóa khi test |
| missingMaSV | SV9999 | Mã chắc chắn không có để kiểm tra 404 |

## Mẫu POST hợp lệ dùng cho test V01

```
POST {{baseUrl}}/api/v1/sinh-vien
Content-Type: application/json

{
  "maSV": "SV006",
  "hoTen": "Nguyễn Minh An",
  "email": "minhan@example.com",
  "maLop": "25CT114"
}
```

## Mẫu PUT hợp lệ dùng cho V06

```
PUT {{baseUrl}}/api/v1/sinh-vien/SV006
Content-Type: application/json

{
  "hoTen": "Nguyễn Minh An Updated",
  "email": "an.updated@example.com",
  "maLop": "25CT114"
}
```

**Lưu ý trước khi test**

Dữ liệu mẫu chỉ phục vụ minh họa. Nếu dùng SQL Server, mã lớp **25CT114** phải tồn tại nếu có khóa ngoại, và SV006 chưa tồn tại trước POST. Chạy trên dữ liệu thử nghiệm, không thao tác nhầm dữ liệu thật.

BƯỚC 8 · DANH SÁCH TEST

# 14 · Bảng kiểm thử Validation bắt buộc

Chạy đúng thứ tự, lưu cả Status thực tế và bằng chứng dữ liệu không bị thay đổi khi Request bị từ chối.

| Mã | Request / tình huống | Kỳ vọng | Xác minh thêm |
|---|---|---|---|
| V01 | POST SV006 đủ 4 trường hợp lệ | 201 | GET đọc được SV006 |
| V02 | POST thiếu hoTen, mã SV khác | 400 | Không thêm sinh viên |
| V03 | POST email không đúng định dạng | 400 | errors có field=email |
| V04 | POST có thêm role: "admin" | 400 | Từ chối trường lạ |
| V05 | PUT SV006 thiếu email | 400 | GET thấy dữ liệu cũ |
| V06 | PUT SV006 đủ 3 trường | 200 | GET thấy dữ liệu mới |
| V07 | PUT SV006 email sai kiểu (số) | 400 | Không ghi dữ liệu sai |
| V08 | POST maSV đã tồn tại | 409 | Không tạo trùng |
| V09 | PUT mã không tồn tại, body hợp lệ | 404 | Không tạo thêm sinh viên |
| V10 | GET /api/v1/sinh-vien | 200 | Danh sách vẫn là mảng |
| V11 | GET /health và kiểm tra logs | Status API cũ | access.log có dòng mới |
| V12 | Restart server, gọi GET lần nữa | Status API cũ | access.log ghi nối |

**Bài kiểm thử hồi quy (Regression Test)**

Sau khi chạy V01–V12, **chạy lại 12 ca T01–T12 của Chương 04** trong đúng điều kiện dữ liệu ban đầu. Các chức năng GET/POST/PUT/DELETE và status 200/201/400/404/409 phải tiếp tục hoạt động.

BƯỚC 8 · ĐỌC KẾT QUẢ

# 15 · Test tự động và kiểm chứng dữ liệu

Mục tiêu không chỉ là nhìn HTTP 400. Sinh viên phải chứng minh Request bị chặn trước Controller và không làm thay đổi bản ghi.

## Trong Postman  Scripts  Post-response, đặt ví dụ:

```
pm.test('Status phải là 400', function () {
  pm.response.to.have.status(400);
});

const body = pm.response.json();
pm.test('Đúng mẫu phản hồi lỗi', function () {
  pm.expect(body.success).to.eql(false);
  pm.expect(body.data).to.eql(null);
  pm.expect(body.errors).to.be.an('array');
});
```

## Muốn kiểm tra đúng field sai (ví dụ email):

```
pm.test('Có lỗi tại email', function () {
  pm.expect(body.errors.some(e => e.field === 'email')).to.eql(true);
});
```

## Cách kiểm chứng V05 không làm hỏng dữ liệu

- GET SV006 trước PUT lỗi  ghi email cũ (A).
- PUT SV006 thiếu email  HTTP 400.
- GET SV006 sau PUT lỗi  email vẫn là A và các trường khác không đổi.
- Ghi bằng chứng vào **docs/test-results.md**, không sửa Expected thành Actual để tạo kết quả PASS giả.

**Mẫu một dòng nhật ký test**

V05 | PUT /api/v1/sinh-vien/SV006 | Body thiếu email | Expected 400 | Actual 400 | GET trước/sau giống nhau | PASS | ảnh minh chứng + mô tả.

BƯỚC 9 · HIỂU AUTH, CHƯA TRIỂN KHAI JWT

# 16 · Phân tích logic kiểm tra quyền theo chương trình Liên thông

Buổi 06 yêu cầu hiểu Auth và để AI hỗ trợ viết logic check quyền, nhưng JWT/bcrypt và đăng nhập thực tế là Buổi 07. Phần này chỉ là bài tư duy và đọc code.

## Quy tắc giả định của hệ thống học vụ

| Người dùng đã xác thực | Có được sửa sinh viên? | Kết quả |
|---|---|---|
| admin | Có | Cho phép đi tiếp |
| student | Không | 403 Forbidden |
| Chưa xác thực | Không | 401 Unauthorized |

## Ví dụ logic nghiệp vụ thuần túy (không phải Auth Middleware)

```
type Role = 'admin' | 'student';

export const canManageSinhVien = (role: Role): boolean => {
  return role === 'admin';
};

// canManageSinhVien('admin')   => true
// canManageSinhVien('student') => false
```

Trả lời trên giấy: Nếu Middleware đã xác minh danh tính hợp lệ, logic nào kiểm tra role? Vì sao không được tin role do người dùng tự gửi trong req.body hoặc tự đặt trong header?

**PROMPT 6 — PHÂN TÍCH CHECK QUYỀN**

**C — Context**: Tôi học Buổi 06 về Middleware và đã có API SinhVien; Buổi 07 mới triển khai JWT.

**T — Task**: Giải thích cách một Middleware tương lai kiểm tra quyền admin trước PUT/DELETE.

**C — Constraint**: Chưa gắn vào API thật; không tin role từ req.body/header của Client; chỉ dùng role đã được Authentication xác minh.

**O — Output**: Trả bảng 401/403, pseudocode có next() và 3 kịch bản minh họa, chưa sinh hệ thống JWT.

KỸ NĂNG AI · CTCO

# 17 · Prompt riêng cho Schema, Middleware và Router

Mỗi prompt xử lý một bước. Sinh viên cần đọc code AI tạo ra, kiểm tra sự khác biệt (diff), chạy build và xác nhận Request/Response thật.

### PROMPT 2 — TẠO ZOD SCHEMA

**C — Context**: Tôi tiếp tục backend-quan-ly-sinh-vien Chương 04 bằng Express/TypeScript, có 5 API và nguồn dữ liệu hiện tại.

**T — Task**: Tạo src/validations/sinhVien.schema.ts gồm createSinhVienSchema và updateSinhVienSchema.

**C — Constraint**: POST yêu cầu 4 trường; PUT chỉ hoTen/email/maLop; trim chuỗi, từ chối trường lạ; không đổi DB, URL hay kiến trúc.

**O — Output**: Trả code đúng đường dẫn, comment tiếng Việt, giải thích từng rule và ví dụ body hợp lệ/sai.

### PROMPT 3 — TẠO VALIDATION MIDDLEWARE

**C — Context**: Tôi đã có hai Zod Schema từ bước trước và Response lỗi của Chương 04.

**T — Task**: Tạo src/middlewares/validate.middleware.ts dùng safeParse để chặn body sai.

**C — Constraint**: Giữ success/message/data; thêm errors (field/message) tùy chọn; HTTP 400 khi sai, next() khi đúng; không viết lại Controller.

**O — Output**: Trả code TypeScript, vị trí file, mô tả req.body và cách kiểm chứng bằng một Request POST sai.

Kiểm tra sau từng Prompt: (1) đúng file? (2) build không lỗi? (3) response có đúng status? (4) dữ liệu có giữ nguyên khi gửi body sai?

KỸ NĂNG AI · CTCO

# 18 · Prompt cho Logging, Swagger và sửa lỗi

Prompt viết rõ bối cảnh đang có sẽ giúp AI không tạo project, Router hoặc file Swagger trùng. Dùng các prompt này sau khi hoàn thành Schema/Middleware.

### PROMPT 4 — GẮN MIDDLEWARE VÀO ROUTER

**C — Context**: API Chương 04 đã khai báo POST/PUT trong src/routes/sinhVien.routes.ts.

**T — Task**: Chỉ ra cách bổ sung validate(createSchema) và validate(updateSchema) trước đúng Controller.

**C — Constraint**: Không tạo router/app mới; giữ GET/DELETE và các import ESM/NodeNext hoặc CommonJS hiện hành.

**O — Output**: Trả diff đoạn code cần thay, không gửi toàn bộ source không liên quan, kèm 3 request kiểm tra.

### PROMPT 5 — MORGAN VÀ OPENAPI YAML

**C — Context**: Tôi có Express app, docs/openapi.yaml và Swagger UI /api-docs từ Chương 04.

**T — Task**: Đề xuất bổ sung Morgan access.log (append) và mô tả errors Validation vào YAML hiện tại.

**C — Constraint**: Không log mật khẩu/token, không tạo Swagger JSDoc hay file openapi khác, giữ response contract cũ.

**O — Output**: Liệt kê file cần thay, phần code thêm tối thiểu, vị trí app.use và cách kiểm chứng file log/Swagger.

### PROMPT DEBUG — KHI KẾT QUẢ KHÁC MONG ĐỢI

**C — Context**: Tôi đang ở Bước [số bước] của Chương 05; project hiện tại là Express + TypeScript.

**T — Task**: Phân tích request/lệnh chạy và lỗi thực tế: [dán log đã bỏ bí mật].

**C — Constraint**: Không đổi kiến trúc, DB, endpoint hoặc package manager; không tự đặt file chưa có.

**O — Output**: Trả nguyên nhân khả dĩ, thay đổi nhỏ nhất theo file/dòng, và test xác nhận sau khi sửa.

XỬ LÝ SỰ CỐ

# 19 · Lỗi thường gặp khi thực hành

Thử xác định lỗi thuộc tầng nào trước khi nhờ AI. Không yêu cầu AI "tạo lại toàn bộ backend" khi chỉ một Request sai.

| Hiện tượng | Kiểm tra đầu tiên |
|---|---|
| POST thiếu hoTen vẫn được 201 | Đã chèn validate(...) vào **đúng** Route chưa? Có route trùng phía trước? |
| Body đúng vẫn trả 400 | Kiểm tra Content-Type, JSON raw, tên thuộc tính và strict() có từ chối trường thừa. |
| req.body là undefined | express.json() phải đăng ký trước router. |
| PUT báo 404 | Kiểm tra tiền tố app.use, HTTP method, :maSV, mã sinh viên có tồn tại. |
| TypeScript báo lỗi đường dẫn import | Đối chiếu module/NodeNext với import nội bộ .js hoặc cách dự án đang dùng. |
| Không thấy access.log | Đã gắn app.use(accessLogger) trước router? Terminal có chạy ở thư mục gốc? |
| access.log bị mất sau restart | Phải dùng flags: "a", không dùng "w". |
| Swagger không hiển thị sửa đổi | Đúng docs/openapi.yaml? Có lặp components? YAML đúng thụt dòng? |
| Swagger hiện schema đúng nhưng API nhận dữ liệu sai | YAML chỉ mô tả; kiểm tra Zod Middleware thực tế. |
| POST 201 nhưng GET không thấy | Kiểm tra Service/DB đã ghi thật; lỗi không thuộc Validation. |

### Cách dùng AI khi báo lỗi

Gửi: **File liên quan + lệnh/request + Expected + Actual + log lỗi đã ẩn bí mật**. Yêu cầu AI đề xuất thay đổi nhỏ nhất. Sinh viên kiểm tra diff, build và chạy lại test trước khi chấp nhận.

BÀI TẬP CỦNG CỐ

# 20 · Nhiệm vụ tự làm và câu hỏi vấn đáp

Tự viết prompt để nâng cấp một quy tắc kiểm tra, không sao chép hoàn toàn prompt mẫu. Mục tiêu là giải thích được quyết định của AI và kiểm chứng bằng test.

## Bài tập bắt buộc A — Mở rộng Validation

• Yêu cầu AI đề xuất kiểm tra độ dài **maSV** hoặc **maLop**, nhưng phải **đối chiếu độ dài thực tế** trong Model/Database trước khi chốt rule.

• Tạo thêm ít nhất 2 test case (1 hợp lệ, 1 không hợp lệ) và lưu kết quả.

• Giải thích vì sao lỗi trùng maSV vẫn cần kiểm tra ở tầng dữ liệu/nghiệp vụ.

## Bài tập bắt buộc B — Nhật ký Request

• Gọi ba Request GET, POST và PUT; mở file logs/access.log, chỉ ra method, URL và status của từng dòng.

• Restart server, gọi GET một lần nữa; chứng minh file được ghi **nối** và không bị ghi đè.

• Giải thích vì sao không nên ghi mật khẩu, JWT hoặc dữ liệu cá nhân nhạy cảm trong log.

## Câu hỏi giảng viên có thể hỏi

| Câu hỏi | Từ khóa cần giải thích |
|---|---|
| Middleware nằm trước hay sau Controller? | Route, next(), short-circuit |
| safeParse khác parse ở điểm nào? | Kết quả success/issues, tránh throw tự động |
| Khi nào 400, 401, 403, 404, 409? | Sai dữ liệu, chưa xác thực, thiếu quyền, không thấy, trùng |
| Vì sao YAML không thay cho Zod? | Đặc tả vs runtime |
| Làm sao biết Validation không phá API cũ? | Regression test, GET kiểm chứng |
| CTCO giúp AI không viết sai project như thế nào? | Bối cảnh, tác vụ, ràng buộc, đầu ra |

BÁO CÁO THỰC HÀNH

# 21 · Hồ sơ cần nộp và tiêu chí tự đánh giá

Bài được hoàn thành khi sinh viên có mã nguồn chạy thật, log thật, Swagger cập nhật và minh chứng kiểm thử; không chỉ có ảnh AI tạo code.

## Thành phần nộp bài

- Mã nguồn dự án **đang dùng** của Chương 04 với file Zod/Middleware/Morgan đã thêm.
- **docs/openapi.yaml** cập nhật (không tách thành bản khác).
- **Postman Collection** và bảng kiểm thử V01–V12 cùng T01–T12 hồi quy.
- **Ảnh minh chứng** ít nhất V01 (201), V03 (400), V05 (400 + GET bảo toàn), V06 (200), access.log.
- **docs/prompt-log.md**: prompt CTCO, file AI đề xuất, code đã review, build và test thật.
- Giải thích ngắn **5–7 câu** về Middleware, next(), Zod Validation và Morgan.

## Mẫu bảng đánh giá gợi ý (10 điểm)

| Tiêu chí | Điểm |
|---|---|
| Kế thừa project Chương 04, không phá CRUD/Swagger | 2.0 |
| Schema POST/PUT đúng hợp đồng, reject dữ liệu sai | 2.0 |
| Middleware validate hoạt động đúng, Response thống nhất | 2.0 |
| Morgan append và bảo mật log | 1.0 |
| Postman + hồi quy + OpenAPI YAML | 2.0 |
| Prompt CTCO, giải thích và code review | 1.0 |

**Không nộp dữ liệu nhạy cảm**

Không nộp node_modules, .env có mật khẩu, token JWT, dữ liệu cá nhân thật hoặc log chứa thông tin bí mật. Có thể nộp .env.example nếu cần cấu hình mẫu.

PHỤ LỤC · KIỂM TRA NHANH

# 22 · Checklist hoàn thành và kết nối Buổi 07

Sinh viên tự đánh dấu trước khi gửi bài. Nếu một mục không đạt, phải ghi nguyên nhân và cách đã kiểm chứng sau khi sửa.

- Đã mở đúng project **backend-quan-ly-sinh-vien**, không tạo dự án mới.
- Đã xác nhận API cũ vẫn hoạt động trước và sau khi thêm Middleware.
- Đã tạo đúng **src/validations/sinhVien.schema.ts**.
- Đã tạo đúng **src/middlewares/validate.middleware.ts**.
- POST/PUT chạy qua validate trước Controller, GET/DELETE giữ nguyên.
- Body sai trả HTTP 400 và không làm thay đổi dữ liệu.
- Body hợp lệ vẫn nhận 201/200 và GET xác nhận có ghi dữ liệu.
- Đã có **logs/access.log** và đã thử cơ chế append.
- Swagger UI đọc được **docs/openapi.yaml** cập nhật.
- Postman chạy V01–V12 và hồi quy T01–T12.
- Có nhật ký Prompt CTCO, thay đổi code và kết quả kiểm thử.
- Chưa tự ý cài JWT/login hoặc mở route quyền Admin giả lập.

## Kết nối bài học tiếp theo

**Buổi 07 — Authentication (JWT):** Sau khi API đã có Validation và Logging, sinh viên sẽ bổ sung đăng ký/đăng nhập, bcrypt, JWT và Auth Middleware xác minh danh tính. Quyền Admin phải kiểm tra trên danh tính đã xác thực, không tin giá trị do Client tự khai báo.

> **Tóm tắt toàn bộ Chương 05**
>
> **Chương 04** làm API CRUD hoạt động. **Chương 05** thêm trạm kiểm tra dữ liệu và nhật ký Request. **Buổi 07** mới xây dựng danh tính và quyền truy cập thật. Cùng một project được nâng cấp theo từng buổi.